You've only added a few DOM IDs and DOM classes.

And with that the test passes again, and I think you've got this feature in the books. Almost.

Testing for Failure

Failure, of course, is always an option, so you need to test for it. I prefer to do failure-path testing in unit tests rather than end-to-end tests. Success requires the entire system to work in concert. A failure response can usually be isolated to one component. That said, it's often useful to have one end-to-end failure test, especially in cases where the failure is easily causable and visible to a typical user.

I haven't yet talked about the tools that will allow you to fake failure—that requires a mock object package. (See Chapter 7, *Using Test Doubles as Mocks and Stubs*, on page 129.) But in this case it's not hard to create a real failure by adding a validation that you can then not fulfill.

Let's first see what an end-to-end failure test might look like, by seeing what would happen if you allow a user to create a project without a name:

test_first/06/spec/system/add_project_spec.rb

```
it "does not allow a user to create a project without a name" do
  visit new_project_path
  fill_in "Name", with: ""
  fill_in "Tasks", with: "Choose Fabric:3\nMake it Work:5"
  click_on("Create Project")
  expect(page).to have_selector(".new_project")
end
```

This test is similar to the original feature test, except that the name field is "filled in" with an empty string. You could just not include that line, but I find it's better to explicitly show that you're setting the name field to blank rather than hope the default works and is readable. In the validation section, you're looking for a `.new_project` class, which is the DOM class for the new project form. In other words, you're checking that you redirect back to something that allows you to reenter the project.

The test fails because you don't have anything catching the failed save. What you want is for the workflow to indicate that saving the project was not successful and then for the controller to redisplay the form on failure rather than display the index page.

Let's also create a test for this condition at the workflow level. Usually I would not write both of these error tests for a condition that was so simple. If I had